

NGQA: Next-Gen Software Quality Accelerator using AI Agents and LLM Reasoning

Seyed Moein Abtahi
seyedmoein.abtahi@ontariotechu.net
Faculty of Engineering and Applied Science
Ontario Tech University
Oshawa, Ontario, Canada

Akramul Azim
akramul.azim@ontariotechu.ca
Faculty of Engineering and Applied Science
Ontario Tech University
Oshawa, Ontario, Canada

Abstract

This study introduces the Next Generation Quality Accelerator (NGQA) pipeline, a comprehensive framework that automates software quality assurance by intelligently orchestrating static analysis and large language models (LLMs). The NGQA methodology addresses the critical challenge of scalable quality assurance by establishing a systematic, six-step pipeline that progressively refines code quality while minimizing human intervention. The framework integrates SonarQube-based issue detection, retrieval-augmented generation (RAG)-driven false-positive mitigation, LLM-based code remediation, structural dependency analysis, comprehensive test suite generation via a novel Local Chain-of-Thought framework that employs four sequential AI agents, and multi-metric quality validation. Experimental evaluation across 70 repositories spanning seven programming languages demonstrates significant effectiveness: PassRatio improved by 16.5%, CodeBLEU increased by 28.8%, and CodeScore improved by 24.0%. The false-positive mitigation agent achieved an 89.0% F1-score, NGQA successfully resolved 83.5% of validated issues, and achieved a 32.6-fold mean acceleration over the estimated manual QA effort, representing a significant step toward largely automated software QA with minimal human intervention.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; *Software evolution*; • **Computing methodologies** → **Artificial intelligence**.

Keywords

Software Quality Assurance, Large Language Models, AI Agents, Static Analysis, Retrieval-Augmented Generation, Local Chain-of-Thought Reasoning, Code Revision, False Positive Mitigation

ACM Reference Format:

Seyed Moein Abtahi and Akramul Azim. 2026. NGQA: Next-Gen Software Quality Accelerator using AI Agents and LLM Reasoning. In *22nd International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '26)*, July 5, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3803846.3807468>



This work is licensed under a Creative Commons Attribution 4.0 International License. *PROMISE '26, Montreal, QC, Canada*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2584-5/26/07

<https://doi.org/10.1145/3803846.3807468>

1 Introduction

Modern software engineering faces substantial quality verification challenges as codebases grow increasingly complex and incorporate multiple programming paradigms within unified projects. This complexity generates an expanding array of potential defects that surpass the capabilities of human-driven inspection processes. The shift toward agile practices and CI/CD workflows has transformed quality verification expectations, requiring automated mechanisms that can keep pace with rapid deployment cycles while delivering consistent and thorough evaluations. Conventional verification strategies relying on manual inspection and human-orchestrated testing exhibit inherent constraints when confronting current development requirements. These approaches suffer from limited throughput, inconsistent application, and vulnerability to oversight. While static code analysis tools offer structured defect identification, traditional detection approaches often fail to recognize nuanced, context-sensitive issues embedded within modern code. Additionally, correction workflows remain largely manual, demanding substantial engineering effort and creating opportunities for incomplete or inconsistent fixes. These bottlenecks impair delivery schedules, compromise system reliability, and diminish development efficiency, particularly as software penetrates critical sectors such as medical systems, banking infrastructure, and essential services.

Emerging capabilities in large language models (LLMs) offer promising avenues for tackling these obstacles. Such neural systems exhibit strong aptitude for parsing intricate code architectures and producing contextually suitable corrections. Combining LLM-powered systems with established defect identification tools enables more sophisticated, adaptable verification mechanisms that can automate both detection and correction workflows. However, significant shortcomings remain where software verification intersects with AI-enabled automation. Existing investigations primarily target individual improvement operations rather than holistic, fully automated workflows. Notable deficiencies include limited understanding of how LLM-based methods perform across varied programming languages and insufficient solutions for privacy-constrained or computationally limited environments unable to leverage commercial cloud platforms. This investigation examines four research questions:

RQ1: Can LLMs effectively automate code revision when guided by static analysis issue detection?

RQ2: How can Chain-of-Thought (CoT) reasoning be effectively leveraged within local, open-source small-scale language models to enable comprehensive test generation, and what advantages does

this multi-step reasoning approach provide compared to a single model?

RQ3: How does the performance of AI-driven quality improvement vary across different programming languages, and what language-specific factors influence the effectiveness of automated code revision and test generation?

RQ4: How effective is a fully integrated AI-driven pipeline in automating the entire software quality improvement lifecycle from issue detection through code revision to validation, compared to traditional manual and semi-automated methods?

To address these inquiries, the NGQA (Next-Generation Quality Accelerator) framework is introduced as a cohesive, automated system that integrates static analysis, false positive elimination, LLM powered code correction, structural mapping, and dual phase test creation. The framework's key innovation involves comprehensive automation across all verification stages, substantially reducing manual involvement while preserving adaptability across diverse languages and project architectures. A Local Chain of Thought (LCoT) strategy is employed to facilitate context sensitive reasoning in open source models through Retrieval Augmented Generation (RAG) integration. A six step workflow is implemented that incrementally enhances code quality: static analysis based defect identification, intelligent false positive removal, automated correction, structural dependency mapping, comprehensive LCoT driven test creation, and thorough validation. This unified design ensures each stage operates with appropriate contextual awareness, resulting in a verification process that maintains robustness, scalability, and substantial autonomy.

2 Background and Related Works

This section examines the theoretical foundations and prior research underpinning the proposed methodology pipeline. The evolution of software quality assurance techniques, the application of large language models in code-related tasks, and recent advancements in automated testing are reviewed to contextualize the contributions of this work.

Modern software quality assurance pipelines integrate static and dynamic analyses with machine-learning-based detection to surface defects earlier in development. Tools like SonarQube continuously analyze code in IDEs and CI/CD environments, flagging vulnerabilities and coding issues before deployment [14, 17, 23]. Static analyzers provide fast, non-executive checks by scanning code against rule sets that encode style, correctness, and security policies, helping developers remediate violations efficiently [11, 24, 34, 47, 48]. LLMs such as GPT-4o and LLaMA demonstrate strong reasoning and multi-step generation capabilities, making them suitable for software tasks [21, 32, 44, 50]. Systems like ChatGPT can draft or refactor code across languages, but their outputs require quality validation through static analysis [9, 39]. Empirical studies show LLM-assisted workflows can accelerate code review and comprehension, though limitations in context-awareness and readability persist [15, 16, 28, 37, 40, 51]. Recent research has explored integrating LLMs with static code analysis to automate software quality improvement. A recent study introduced a pipeline in which static analyzers detect code issues, and LLMs generate context-aware

fixes enhanced through retrieval-augmented generation (RAG), effectively reducing bugs, vulnerabilities, and code smells across projects [1].

Retrieval-Augmented Generation (RAG) enhances LLM performance by fetching external knowledge (e.g., API docs, commit history) during inference and conditioning responses on that context [6, 10, 54]. Coupling retrieval with generation improves factual grounding, reduces hallucinations, and boosts relevance in domain-specific scenarios [19, 25, 43, 52]. Evaluating AI-generated code requires both automated and human-centered metrics [1]. Structural quality is assessed through static rule violations, defect detection, and security compliance [9, 47], while readability influences developer trust [37]. Effective evaluation should also capture functional correctness, syntactic similarity, and semantic fidelity [36]. Multidimensional frameworks that incorporate robustness, usability, and productivity enable systematic comparisons of LLM-assisted workflows against traditional development practices. Automated test generation has evolved from early data-driven methods to AI-powered approaches that support adaptive and self-healing tests for agile pipelines [3, 4, 33]. LLM-based tools now automate full class- and project-scale suites. Frameworks such as AGONETEST enable end-to-end generation and validation [27], while CHATUNITEST integrates context adaptation and repair mechanisms to outperform traditional baselines like EvoSuite [8].

Chain-of-thought (CoT) prompting encourages large language models to externalize stepwise reasoning, improving performance on complex tasks. Recent work distinguishes "Long CoT" characterized by deeper exploration from shorter explanations, linking Long CoT to gains in advanced reasoning models [7]. Pattern-aware prompting enhances robustness by selecting demonstrations that reflect diverse reasoning strategies [53]. Broader surveys highlight the role of CoT traces in coordination, tool use, and benchmark evaluation, yet note the need for standardized taxonomies [18]. Traditional Abstract Syntax Tree (AST)-based tools rely on deterministic pattern matching but often miss 47–80% of real-world vulnerabilities in C/C++ code [26]. A 2025 benchmark shows industry-standard SAST tools like SonarQube and CodeQL achieved F1 scores of 0.26–0.54, while LLMs reached 0.75–0.80 [20]. Recent frameworks leverage this complementary relationship: ZeroFalse employs LLMs as post-filters to prune false alarms, achieving F1-scores up to 0.95 [22]. Similarly, the NGQA pipeline combines SonarQube's structural detection with LLM-based reasoning, achieving an 89.0% F1-score in filtering and an 83.5% overall issue resolution rate.

3 Methodology

The NGQA (Next-Generation Quality Assurance) pipeline is a framework for automating software quality assurance through the integration of static analysis and large language models (LLMs). It addresses scalable quality assurance via a systematic six-step process that progressively improves code quality with minimal human intervention. The pipeline comprises static issue detection, false-positive mitigation, LLM-based code revision, structural dependency analysis, comprehensive test suite generation, and multi-metric quality validation. While each step can be applied independently for specific quality assurance tasks, their combined orchestration enables

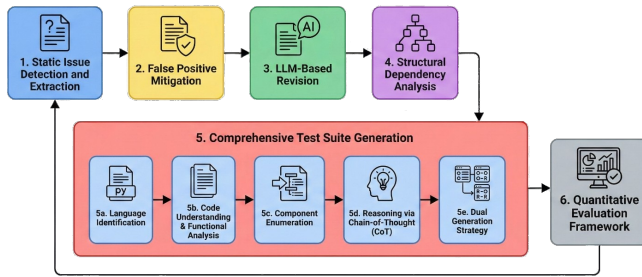


Figure 1: The NGQA Pipeline for Automated Software Quality Assurance

accelerated and near-complete automation of the quality assurance lifecycle, as illustrated in Fig. 1.

3.1 Static Issue Detection and Extraction

Since this work is centered on automation, each step of the NGQA pipeline is implemented using specialized tools and techniques. In the initial step, static analysis tools are used to detect issues within the software project automatically. The input to the pipeline is human-written code, which serves as the foundation for both the subsequent development stages and the final evaluation. To ensure broad applicability, the pipeline is designed to support multiple programming languages. Static analysis tools identify issues based on language-specific rules and patterns, as well as known bugs and best practices curated by the developer and open-source communities.

3.2 False Positive Mitigation

One common limitation of static analysis tools is the generation of false positive instances where non-erroneous lines of code are incorrectly flagged as issues. To ensure the accuracy of the subsequent steps in the pipeline, it is essential to evaluate and filter out these false positives. In this step, an AI agent is employed to identify and flag potential false positives among the issues detected in the previous step. This agent operates early in the pipeline to eliminate incorrectly reported issues before they are further processed. To enhance its decision-making capabilities, the agent is equipped with a Retrieval-Augmented Generation (RAG) mechanism. This allows it to access and leverage external knowledge sources such as GitHub repositories, Stack Overflow discussions, and Google search results, thereby improving the precision of false positive identification and supporting the language model’s evaluation process. The RAG mechanism contributes to false positive reduction through evidence-based verification. When evaluating issues detected by the Issue Detection stage, the agent queries external knowledge sources using the issue description, rule identifier, and code context. The retrieved information, including similar issue reports, community discussions, and resolution approaches, helps the agent and LLM understand which flagged issues are not genuine problems. If external sources provide evidence that a particular flagged pattern is actually acceptable or safe in the given context, the agent classifies it as a false positive. However, if RAG retrieval yields no relevant details contradicting the static analysis finding, the issue

remains classified as legitimate. This approach uses the Issue Detection tool’s output as the baseline, with external knowledge serving to identify and filter out incorrectly flagged issues.

3.3 LLM-Based Revision

Following the identification of code issues and the elimination of false positives, the pipeline proceeds to the code refinement step, which is facilitated by a specialized AI agent powered by LLMs. This agent is tasked with systematically revising the original source code to address the previously detected issues in an automated and context-sensitive manner.

To enhance the accuracy and domain-specific relevance of the revisions, the agent integrates external knowledge sources, such as GitHub repositories and SonarQube community discussions, through the use of RAG techniques. This methodological integration ensures that the model’s output is informed not only by its pre-trained knowledge but also by up-to-date and community-validated programming practices and solutions. A central challenge in utilizing LLMs for automated code modification is the propensity for generating hallucinated outputs, which may appear syntactically and semantically plausible but lack grounding in the input data. To mitigate this limitation, the proposed pipeline incorporates rigorous validation procedures, including the automated execution of test cases and the application of evaluation metrics. These mechanisms serve to assess the functional correctness and overall quality of the revised code before it is included in the final software system.

3.4 Retrieval Augmented Generation

Retrieval-augmented generation (RAG) integrates an information-retrieval step that connects AI agents to external knowledge sources, including GitHub repositories, Stack Overflow discussions, SonarQube resources, and Google Search, enabling access to current solutions beyond the model’s training data. The RAG pipeline operates through a hierarchical retrieval strategy. AI agents construct domain-specific search queries from issue descriptions and code context, then systematically query specialized technical sources such as GitHub, Stack Overflow, and SonarQube before expanding to general web search if necessary. This tiered approach prioritizes authoritative technical resources while ensuring comprehensive information coverage. Retrieved information is synthesized and ranked by source credibility and contextual relevance. The prioritized results are integrated into the agent’s knowledge context before LLM processing, grounding responses in verified real-world solutions rather than pre-trained knowledge alone. When external sources provide insufficient information, the system gracefully defaults to the model’s inherent capabilities, maintaining response quality without degradation. The RAG process is shown in Fig. 2. By grounding LLM outputs in verified external sources, our RAG-enhanced pipeline significantly improves code revision accuracy, reduces hallucination risks, and provides contextually aware recommendations [1, 54].

3.5 Structural Dependency Analysis

To enhance the effectiveness of test case generation in Section 3.7, particularly for integration testing, it is essential to understand the structural relationships among source files within the software

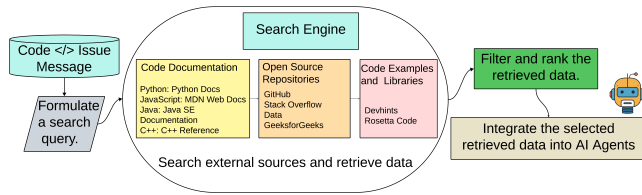


Figure 2: Retrieval-Augmented Generation process

project. Integration testing involves combining individual modules and evaluating them collectively to ensure correct interaction and system-wide functionality. Constructing a comprehensive project map that represents the hierarchical organization and interdependencies of files provides valuable context for this purpose. LLMs can leverage this structural representation during the test case generation step to produce more accurate and contextually relevant test scenarios. By incorporating project mapping, the pipeline facilitates a deeper semantic understanding of the codebase, thereby improving the quality and precision of the generated integration tests.

3.6 AI Agents

AI agents play a central role in the NGQA pipeline, orchestrating various steps of automation and decision-making. These agents are responsible for performing retrieval-augmented generation (RAG), preparing structured input for subsequent agents within a chain-of-thought (CoT) reasoning framework, and generating context-aware responses based on the input data. By leveraging these capabilities, AI agents enable seamless coordination across the pipeline, ensuring that each component operates with relevant context and that the overall QA process remains robust, scalable, and largely autonomous. The internal operations of AI agents, including their specific roles and operational details, are described in subsection 4.3.5.

3.7 Comprehensive Test Suite Generation

Test case generation presents a greater challenge than code revision. As discussed in Section 3.3, code revision is guided by the source code and specific issues identified through static analysis. Leveraging LLMs and code revision agents, the pipeline can effectively refactor the code to address these issues. However, test case generation lacks a comparable baseline, as we assume no pre-existing human-written tests are available. This assumption ensures the pipeline's applicability across a wide range of software projects and programming languages. As shown in Fig. 1 step 5, LLMs must independently generate test cases based on their interpretation of the code. The process begins with identifying the programming language of the source file, followed by analyzing the code to understand the functionality of each unit. The model then enumerates the components within the file, such as functions, classes, and logic blocks, and extracts top-level definitions, including their names, arguments, and docstrings when available.

The test generation process employs a white-box testing approach with full visibility into code structure, logic paths, and implementation details. However, without access to requirements specifications or functional documentation, the generated tests are constrained to validating implementation consistency rather

than verifying correctness against intended specifications. The LLM infers expected behavior from code structure, variable names, docstrings, and common patterns, but cannot validate whether the implementation fulfills its intended purpose. For each component, the LLM generates unit tests that encompass normal cases reflecting standard usage, edge cases involving boundary values, empty inputs, or nulls, and invalid cases characterized by incorrect data types or missing arguments. These tests verify that the code behaves consistently with its apparent intent, but may miss functionality-level defects if the implementation itself deviates from requirements.

Furthermore, the project mapping step described in Section 3.5 provides critical context by supplying a structural overview of the project and inter-file dependencies. This global perspective enables the LLM to generate more coherent and contextually appropriate test cases, especially for integration testing. This multistep reasoning process follows the Chain-of-Thought (CoT) prompting methodology, which is particularly effective when applied with high-performing models such as OpenAI's O1, O3-mini, and DeepSeek-R1 [42]. These models, often referred to as "reasoning models," are designed to simulate structured analytical reasoning. In this study, CoT and reasoning strategies are employed using more than four distinct LLMs, including those without inherent reasoning capabilities, and augmented by AI agents to maximize the utility and quality of the generated test cases.

By leveraging AI agents, multiple LLMs, RAG techniques for incorporating external domain knowledge, and prompt engineering strategies, this pipeline facilitates the generation of high-quality test cases that serve as a foundation for robust evaluation. A critical insight is that a single round of test case generation is insufficient for comprehensive assessment. Therefore, the pipeline generates two distinct sets of test cases: one based on the source code and another based on the LLM-revised version of the code.

This dual-generation strategy enables a four-fold testing process, where each test set is executed against both the original and revised codebases, allowing for robust comparative analysis. To ensure test quality, the pipeline validates that generated tests contain substantive executable statements and execute deterministically against the target code before inclusion in the evaluation suite. By identifying discrepancies and measuring improvements across these iterations, the pipeline progressively converges on the most accurate version of the code and its corresponding test suite, enabling detailed comparative evaluation and iterative refinement that strengthens the integrity of LLM-driven code revision.

3.8 Evaluation Metrics

Tests that pass successfully after code revision serve as an indicator of improvement when compared against the original buggy code. However, PassRatio [13] alone is insufficient to assess overall code quality. Therefore, additional evaluation metrics are incorporated to provide comprehensive assessment of defect reduction, structural quality, and execution correctness.

The PassRatio metric quantifies the proportion of test cases for which the generated code produces correct outputs. As formalized in Equation 1, it is computed by evaluating the generated code g against a set of test cases C , where each test case c consists of input I_c and expected output O_c . The indicator function $\mathbb{I}$ returns 1 when

the evaluation function $\text{Eval}(g, I_c)$ produces output matching O_c , and 0 otherwise. PassRatio improvements directly correlate with reduced runtime failures and improved software reliability.

$$\text{PassRatio} = \frac{1}{|C|} \sum_{c \in C} \mathbb{I} \{ \text{Eval}(g, I_c) = O_c \} \quad (1)$$

The *CodeBLEU* [41] metric extends BLEU [35] by incorporating programming-specific structural and semantic features. As shown in Equation 2, it combines four components: standard BLEU for lexical similarity, weighted BLEU ($\text{BLEU}_{\text{weight}}$) for key tokens, abstract syntax tree matching ($\text{Match}_{\text{ast}}$) for structural similarity, and data flow matching (Match_{df}) for semantic relationships. Higher CodeBLEU scores indicate improved code organization and maintainability. However, it evaluates structural and syntactic similarity rather than functional correctness, and should not be interpreted as proof of semantic equivalence.

$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU}_{\text{weight}} + \delta \cdot \text{Match}_{\text{ast}} + \zeta \cdot \text{Match}_{\text{df}} \quad (2)$$

CodeScore [13] leverages a LLM to estimate functional correctness by learning from simulated execution behavior. Unlike traditional metrics that rely on lexical similarity, CodeScore predicts how well the generated code would perform across test scenarios. The model UniCE returns a continuous correctness score along with an executability flag:

$$\text{CodeScore, Exec} = \text{UniCE}(x) \quad (3)$$

where x represents the combined input sequence. CodeScore captures subtle logical errors that might pass syntactic checks but produce incorrect runtime behavior, reducing the probability of production failures.

To automate the evaluation process, an AI agent equipped with these metrics computes evaluation scores for each revised file during every QA cycle, determining whether a given file requires further revision or can be finalized. Having outlined the architecture, components, and processes of the NGQA pipeline, we now turn to its empirical validation. The next section describes the experimental setup, detailing the tools, datasets, and implementation environment used to operationalize the proposed framework and evaluate its effectiveness.

4 Experiment Plan

This section provides an overview of the configuration and resources used in implementing and evaluating the NGQA pipeline. The software tools, language models, and external knowledge sources integrated into the workflow are described. All tools, models, and services presented in this section represent one concrete instantiation of the pipeline; NGQA is technology-agnostic, and any component can be replaced with suitable alternatives based on the user’s infrastructure and requirements. A comprehensive implementation, including all source files and the complete n8n workflow, has been made publicly available in the anonymized repository¹.

¹<https://github.com/MoeinAbtahi/Next-Gen-Software-Quality-Accelerator-AI-Agents-with-LLM-Reasoning>

Table 1: Dataset Overview and Issue Distribution by Programming Language

Language	# Repo	Total LOC	Total Files	Bugs	Code Smells	Vul. Issues
TypeScript	10	195,800	1,420	102	334	47
JavaScript	10	154,200	890	85	279	31
Python	10	251,500	1,345	156	478	72
Java	10	318,900	2,010	189	567	86
C++	10	187,600	1,340	134	421	63
C#	10	263,400	1,780	148	485	74
C	10	139,200	980	91	287	39
Total	70	1,510,600	9,765	905	2,851	412

4.1 Dataset and Issue Distribution

The NGQA pipeline is designed to automate quality assurance across a broad spectrum of programming languages that support test case integration. To evaluate its generalizability and effectiveness, we selected 70 open-source GitHub repositories, with ten repositories chosen for each of seven programming languages: JavaScript, TypeScript, Python, Java, C, C++, and C#. The selected languages exhibit inherent differences in type systems and behaviors that naturally affect quality assurance outcomes. Dynamically-typed languages like Python and JavaScript permit more lenient runtime behavior, while statically-typed languages like C++ and C impose strict compile-time constraints and memory management requirements. Consequently, equivalent code quality levels may correspond to different quantitative metrics across languages, independent of pipeline effectiveness.

Repositories were randomly selected from publicly available open-source projects on GitHub, prioritizing those with a minimum of 10 SonarQube-detected issues to provide sufficient evaluation data. This strategy ensures diversity in code quality, project maturity, and programming practices while maintaining balanced language representation. The dataset includes source files in formats such as .js, .jsx, .ts, .tsx, .mjs, .py, .java, .cs, .c, .cpp, .cxx, and .cc. Table 1 summarizes the aggregated characteristics by language. The dataset comprises 1,510,600 lines of code across 9,765 files, with SonarQube identifying 4,168 issues in total: 905 bugs (21.7%), 2,851 code smells (68.4%), and 412 vulnerabilities (9.9%). This distribution reflects typical patterns in real-world projects, where code smells constitute the majority of quality issues. Java repositories contributed the largest codebase (318,900 LOC) and highest issue count, while C repositories represented the smallest subset, facilitating comprehensive cross-language analysis.

4.2 Baseline Pipeline and Comparative Framework

In this paper, comparisons are established based on the Automated Code Issue Detection and Revision Pipeline [1], a state-of-the-art framework that employs a two-tier LLM approach (GPT-3.5 Turbo and GPT-4o) for integrated static analysis and code revision [2]. To enable rigorous comparison, we re-implemented the baseline pipeline using NGQA’s standardized evaluation metrics on an identical dataset of 70 open-source repositories across seven programming languages. Both pipelines were evaluated using PassRatio, CodeBLEU, CodeScore, and F1-score metrics. The baseline results serve as the comparative reference point throughout this paper, with all reported NGQA improvements calculated relative to the baseline pipeline’s performance on identical datasets and metrics.

4.3 Pipeline Orchestration and Execution

To implement the steps of the NGQA pipeline, we employed the open-source workflow automation tool *n8n* (short for “nodemation”) [29]. This platform offers a flexible environment for orchestrating AI agents, integrating large language models (LLMs), and automating interactions with external services. When deployed locally, *n8n* supports executing Python scripts, issuing HTTP requests, web crawling, and interacting with command-line utilities.

Our experimental setup combines cloud-based and locally deployed language models. For cloud-based inference, we utilize models from OpenAI [31]. For local processing, we deploy LLMs via the *Ollama* framework [30], enabling efficient and privacy-preserving execution. This hybrid architecture offers flexibility and tighter control over data flow. The NGQA pipeline integrates with GitHub for accessing source code repositories.

4.3.1 SonarQube Static Code Analysis. To systematically identify and evaluate issues in the software codebase, we utilized *SonarQube* [46], an open-source platform developed by SonarSource. *SonarQube* performs continuous static code analysis, providing detailed insights into various dimensions of code quality. It supports a broad range of programming languages and incorporates quality gates validation mechanisms that ensure adherence to predefined coding standards and metrics.

Following issue detection with *SonarQube*, a Python script extracts and organizes the identified issues into a structured format compatible with the subsequent pipeline components. An example of the resulting CSV file is presented in Table 2. Extracted issues are grouped into distinct categories based on *SonarQube*’s classification system: **bugs** (coding errors that may lead to incorrect program behavior, including compilation issues, logic errors, and unintentional mistakes introduced during development; note that this term follows *SonarQube*’s definition and encompasses a broader scope than the traditional software engineering definition of bugs as faults revealed through specific code execution), **code smells** (signs of potential design or structural issues that may indicate the need for refactoring to improve maintainability), and **vulnerabilities** (security-related weaknesses in the software that could be exploited by malicious actors).

4.3.2 RAG-Enhanced Issue Filtering. Following static issue extraction, a dedicated AI agent equipped with RAG, as introduced in Section 3.4, identifies and removes false positives from the initial CSV file, producing a refined dataset for subsequent pipeline stages. The agent queries external sources, ranking retrieved results by source credibility, contextual relevance, cross-source consistency, and community validation before integrating them into the agent’s prompt context. Because retrieval draws from live external sources, evolving community knowledge is automatically incorporated without requiring pipeline modifications, distinguishing NGQA from static approaches bounded by fixed training data.

4.3.3 LLM Revision. The third step of NGQA begins with the user providing the path to a filtered CSV file (produced in 4.3.2) containing static analysis results generated by *SonarQube*. A Python script then initializes the required output directory structure, automatically creating subdirectories based on the file metadata. The CSV file is read and transformed into a format compatible with *n8n*’s

workflow system. The parsed data is then processed in a loop that invokes multiple AI agents for code revision tasks, one per source file [1]. After the revision step, the workflow transitions to test case generation.

4.3.4 Structural Dependency Extraction. Prior to test case generation, a Python script extracts a structural representation of the codebase, referred to as the project tree, capturing the hierarchical organization of files and their interdependencies. When a target file imports modules, inherits classes, or calls functions from other files, the project tree provides the model with access to those connected components. This contextual awareness enables the generation of test cases that account for cross-file dependencies, inherited behaviors, and external function calls, rather than treating each file in isolation.

4.3.5 Dual Test Case Generation with LCoT. To approximate reasoning capabilities without relying on proprietary cloud-based models explicitly designed for this purpose, we implemented a sequential configuration of four locally hosted LLMs using the *Ollama* framework [30], referred to as Local Chain of Thought (LCoT). These open-source models include Deepseek [12], LLaMA [49], and Qwen [38]. The decision to employ local models instead of cloud-based alternatives serves multiple purposes. It enables analysis of whether coordinated ensembles of smaller local models (14 to 16 billion parameters) can achieve comparable performance to single large-scale proprietary reasoning systems. Additionally, local models provide a viable solution for processing sensitive data and proprietary codebases in security-critical projects where data privacy regulations prohibit transmission to external cloud services. Task specialization across four agents also produces more focused outputs than prompting a single generalist model for all subtasks sequentially. The model selection within this framework is not fixed; users may substitute any locally deployable models compatible with *Ollama* based on their hardware and task requirements.

This staged architecture supports progressive task decomposition, with each model performing a distinct subtask and passing its formatted output as input to the next, forming a structured, stepwise workflow that simulates a chain-of-thought (CoT) reasoning mechanism. The **first** AI agent, powered by *deepseek-code-v2:16b*, performs functional analysis of the source code and identifies its key components. The **second** agent, also using *deepseek-code-v2:16b*, extracts top-level elements such as functions and classes along with their arguments and docstrings, receiving both the original source code and the output of the previous step as input. The **third** agent, running *codellama:13b*, generates test scenarios for each identified component, incorporating information from the second agent, the source code, and project structure metadata such as file hierarchies and dependencies. The **fourth** and final agent, powered by *qwen2.5-code:14b*, generates unit tests for each identified function or class, covering normal execution paths, edge cases, and invalid input scenarios, while ensuring appropriate exception handling, consistent test naming, and compliance with language-specific syntax conventions.

The selection of these local models was constrained by the available experimental hardware, namely an NVIDIA RTX 3060 GPU with 16GB of memory, which necessitated models in the 12–16 billion parameter range, the largest coding-specialized models capable

Table 2: Example Report Format for Code Quality Issues Detected by SonarQube

File Location	File Name	Line	Message	Type	Output Location	Test Location	Test File Name
.../eval/mbpp/ mbpp.cpp	mbpp.cpp	207	Remove this unexpected named argument 'references'.	BUG	...Rev/eval/mbpp/ mbpp.cpp	...Rev/eval/mbpp/ tests/mbpp.test.cpp	mbpp.test.cpp
.../eval/gsm/ GsmEval.java	GsmEval.java	154	Use of weak cryptographic algorithm (MD5); consider using SHA-256 instead.	VULNER	...Rev/eval/gsm/ GsmEval.java	...Rev/gsm/tests/ GsmEval.test.java	GsmEval.test.java

of running efficiently within these constraints. DeepSeek, CodeL-LaMA, and Qwen were among the most capable open-source coding models meeting these requirements while delivering meaningful performance for code analysis and test generation. All agents within the LCoT framework are enhanced with RAG capabilities, enabling access to external knowledge sources including Stack Overflow, GitHub repositories and community discussions, the SonarQube community, and Google Search, which collectively provide context-aware information to improve response accuracy and reliability.

Building on the LCoT process described, we extend the experiment to incorporate a dual test case generation strategy. In the first pass, test cases are generated based on the original source code. This process is then repeated using the revised code produced through the LLM-based revision step described in Section 4.3.3. This dual-generation approach enables a more comprehensive evaluation framework, allowing us to assess both the quality of the generated test cases and the effectiveness of the code revisions across four distinct execution scenarios.

4.3.6 Four-Fold Testing. Following test case generation, the final step of the NGQA pipeline involves executing the tests to evaluate both the original and revised code versions. Specifically, we utilize four components: the original code (OC), the revised code (RC), test cases generated based on the original code (OT), and test cases generated based on the revised code (RT). This setup enables four distinct test executions: (1) OC with OT, (2) OC with RT, (3) RC with OT, and (4) RC with RT. These four combinations allow for a comprehensive evaluation of test coverage and revision quality.

With the experimental framework, tooling infrastructure, and multi-language dataset established, the NGQA pipeline is evaluated in practice. Results across all study repositories are presented in the following section, with analyses focusing on code revision effectiveness, false-positive mitigation, test case quality, and overall pipeline performance using the metrics defined in Section 3.8.

5 Results and Analysis

This section presents a comprehensive evaluation of the NGQA pipeline’s performance across 70 open-source GitHub repositories, encompassing multiple programming languages and diverse codebases. The analysis focuses on four key aspects: static issue detection effectiveness, false-positive mitigation accuracy, LLM-based code revision quality, and comprehensive test suite generation performance.

5.1 False Positive Mitigation Performance

This subsection addresses RQ4 by evaluating the precision of the RAG-enhanced filtering stage that precedes LLM-based revision, ensuring that the integrated pipeline operates on validated issues rather than raw static analysis output. The AI agent responsible

for identifying false positives demonstrated significant effectiveness in refining the initial dataset of 4,168 detected issues. Table 3 presents the performance metrics across different issue categories and programming languages. The agent achieved an overall precision of 85.6%, recall of 92.6%, and F1-score of 89.0%. C demonstrated the highest precision, with variation across languages within $\pm 3.2\%$. Java showed the lowest precision, attributable to its object-oriented paradigm complexity, extensive use of design patterns, and framework-specific idioms that static analysis tools commonly misinterpret. The agent filtered out 598 false positives (14.4% of initial detections), reducing the issue count from 4,168 to 3,570 valid issues for subsequent processing. The consistent performance across languages validates the robustness of the RAG-enhanced agent in distinguishing genuine quality issues from spurious alerts.

Table 3: False Positive Mitigation Results

Issue Type	Initial Count	False Positives	Precision	Recall	F1-Score
Bugs	905	127	0.859	0.924	0.890
Code Smells	2,851	422	0.852	0.918	0.884
Vulnerabilities	412	49	0.881	0.952	0.915
Overall	4,168	598	0.856	0.926	0.890

5.2 Test Suite Generation and Analysis

This subsection addresses RQ2 by evaluating whether the LCoT framework, built from sequentially orchestrated open-source small-scale LLMs, produces test suites of sufficient quality and coverage compared to single-model approaches. The comprehensive test suite generation component utilizing the LCoT framework produced extensive test coverage across all evaluated repositories. Table 4 summarizes the test generation statistics and the subsequent four-fold testing evaluation. The LCoT framework generated an average of 230 test cases per repository for original code (OT) and 260 test cases for revised code (RT), representing a 13.0% increase in the number of generated test cases for the revised implementations. This increase reflects the enhanced code structure and improved function definitions resulting from the LLM-based revision process. Python repositories generated the highest number of test cases (avg. 319 RT), while C repositories generated the fewest (avg. 167 RT), correlating with codebase complexity and available testing patterns. The four-fold testing evaluation reveals distinct performance patterns across languages:

OC-OT (Original Code with Original Tests): Baseline performance with an average pass rate of 73.8%, with Python showing the highest baseline (76.9%) and C++ the lowest (70.6%). **OC-RT (Original Code with Revised Tests):** Reduced performance (68.2%) due to more stringent test requirements generated for the revised code, with variation ranging from 65.1% (C++) to 71.3% (Python). **RC-OT (Revised Code with Original Tests):** Improved

performance (82.1%) demonstrating the effectiveness of code revision, with Python achieving the highest improvement (85.4%). **RC-RT (Revised Code with Revised Tests)**: Optimal performance (86.0%) representing the best alignment between revised code and corresponding test cases, with Python leading at 90.3%.

The 11.2% improvement from OC-OT to RC-OT validates the effectiveness of the LLM-based revision process in addressing the originally detected issues while maintaining compatibility with existing test requirements. The 16.5% improvement from OC-OT to RC-RT demonstrates the synergistic effect of code revision and enhanced test generation.

Table 4: Test Suite Generation and Four-Fold Testing Results by Language

Language	Generated Tests		Four-Fold Testing Pass Rates			
	OT Count	RT Count	OC-OT	OC-RT	RC-OT	RC-RT
TypeScript	192	218	0.758	0.702	0.842	0.882
JavaScript	156	179	0.751	0.695	0.836	0.879
Python	284	319	0.769	0.713	0.854	0.903
Java	345	389	0.724	0.668	0.807	0.841
C++	218	247	0.706	0.651	0.789	0.825
C#	267	301	0.742	0.686	0.825	0.864
C	147	167	0.713	0.658	0.796	0.829
Average	230	260	0.738	0.682	0.821	0.860
Improvement	+13.0%		-7.6%		+4.8%	

5.3 Code Revision Effectiveness

This subsection addresses RQ1 and RQ4 by quantifying the extent to which static-analysis-guided LLM revision improves code quality relative to the baseline pipeline across all evaluated languages and metrics. The LLM-based code revision component demonstrated substantial improvements in code quality across multiple evaluation metrics. To evaluate the functional correctness of revised code, we utilized the test suites generated through the process described in Section 3.7. Specifically, test cases were generated for both original and revised code versions, enabling comparative assessment of revision effectiveness through the PassRatio metric alongside structural metrics (CodeBLEU) and LLM-based assessment (CodeScore). To establish rigorous baseline comparisons, we evaluated both NGQA and the baseline pipeline [1] using standardized evaluation metrics on our identical dataset of 70 repositories across seven programming languages. Table 5 presents the quantitative assessment of code revision effectiveness using PassRatio (Eq. 1), CodeBLEU (Eq. 2), and CodeScore (Eq. 3) metrics for both pipelines.

Table 5: Code Revision Quality Assessment: NGQA vs Baseline Pipeline

Language	PassRatio		CodeBLEU		CodeScore	
	Baseline	NGQA	Baseline	NGQA	Baseline	NGQA
TypeScript	0.758	0.882	0.635	0.814	0.691	0.853
JavaScript	0.751	0.879	0.628	0.809	0.684	0.847
Python	0.769	0.903	0.647	0.835	0.704	0.869
Java	0.724	0.841	0.591	0.768	0.653	0.812
C++	0.706	0.825	0.579	0.751	0.637	0.796
C#	0.742	0.864	0.618	0.793	0.675	0.839
C	0.713	0.829	0.584	0.748	0.648	0.801
Average	0.738	0.860	0.612	0.788	0.670	0.831
Improvement	+16.5%		+28.8%		+24.0%	

The results demonstrate consistent improvements of NGQA over the baseline across all programming languages and evaluation metrics, as shown in Table 5. PassRatio improved by an average of

16.5% (73.8% to 86.0%), indicating enhanced functional correctness of the revised code. Since test cases are derived from the same codebase, however, this metric reflects implementation consistency rather than validated correctness against external specifications. CodeBLEU improved by 28.8% (61.2% to 78.8%), reflecting better structural alignment through AST matching and data flow analysis, interpreted here as a proxy for maintainability rather than semantic correctness. CodeScore exhibited a 24.0% improvement (67.0% to 83.1%), capturing subtle logical errors that pass syntactic checks but produce incorrect runtime behavior.

Each metric addresses a distinct quality dimension. PassRatio measures implementation consistency and runtime reliability; improvements indicate that the revised code satisfies more test requirements than the original. CodeBLEU reflects structural organization and code clarity, with higher scores suggesting reduced technical debt. CodeScore estimates execution correctness through simulated runtime behavior, reducing the probability of production failures. Together, these three metrics provide complementary coverage of functional, structural, and execution-level quality.

5.4 Issue Type Resolution Analysis

This subsection addresses RQ4 by analyzing resolution rates across bug, code smell, and vulnerability categories within the fully integrated pipeline. The analysis of issue resolution effectiveness across categories reveals varying success rates for bugs, code smells, and vulnerabilities. Following the false positive mitigation step, which filtered out 598 false positives (127 bugs, 422 code smells, and 49 vulnerabilities), the remaining 3,570 validated issues were processed for resolution. Table 6 presents the resolution statistics for each issue type based on the validated issue set. Vulnerabilities achieved the highest resolution rate (90.4%), followed by bugs (83.7%) and code smells (82.4%).

Table 6: Issue Resolution Effectiveness by Category

Issue Type	Total Issues	Resolved	Partial	Unresolved	Success Rate
Bugs	778	651	89	38	83.7%
Code Smells	2,429	2,001	311	117	82.4%
Vulnerabilities	363	328	27	8	90.4%
Total	3,570	2,980	427	163	83.5%

5.5 Language-Specific Analysis

This subsection addresses RQ3 by examining how pipeline effectiveness varies across programming languages and the factors driving these differences. Fig. 3 illustrates the comparative performance across all languages for PassRatio, CodeBLEU, CodeScore, and Issue Resolution Rate improvements. Python achieved the highest performance, benefiting from rich LLM pretraining data and docstring usage, while C and C++ produced the lowest results due to memory management complexity and limited test-rich training data. Overall, pipeline effectiveness is primarily governed by LLM pretraining data richness, type annotation availability, and paradigm complexity.

5.6 Practical Efficiency: Manual QA vs. NGQA

To contextualize NGQA's practical value, we estimated the equivalent manual QA effort using the COCOMO II Post-Architecture model [5, 45] applied to the experimental dataset. Manual QA was

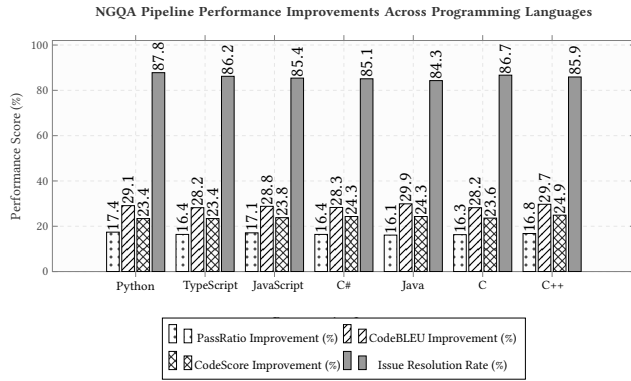


Figure 3: Performance improvements distinguished by patterns for grayscale accessibility.

estimated at 0.75 hrs/issue for review, fix implementation, and validation, while NGQA processes each issue at 0.0139 hrs/issue plus 4.0 hrs of fixed overhead. Across the 4,168 detected issues, this yields an estimated manual effort of **3,126 person-hours** compared to **95.9 hours** for NGQA, achieving a **32.6-fold mean acceleration**. Acceleration varies by language, from 31.2-fold for JavaScript to 40.2-fold for Java, reflecting differences in code complexity and issue density. Table 7 presents the per-language breakdown.

Table 7: NGQA acceleration across programming languages

Language	LOC	Issues	Manual (hrs)	NGQA (hrs)
TypeScript	195,800	483	362.3	10.7
JavaScript	154,200	395	296.3	9.5
Python	251,500	706	529.5	13.8
Java	318,900	842	631.5	15.7
C++	187,600	618	463.5	12.6
C#	263,400	707	530.3	13.8
C	139,200	417	312.8	9.8
Total	1,510,600	4,168	3,126.2	95.9
Mean Acceleration Factor			32.6x	

6 Limitations and Future Work

While the NGQA pipeline demonstrates strong performance in automated quality assurance, several limitations remain. Its effectiveness is bounded by LLM capabilities and residual hallucination risks, despite the use of validation and RAG techniques. Performance varies across programming languages (Python: 90.3% vs. C++: 82.5% PassRatio), reflecting disparities in training data. Sequential processing and the computational cost of multiple LLM agents raise scalability concerns for large codebases, and generated tests may overlook domain-specific edge cases. Although false positive mitigation achieves an 89.0% F1 score, remaining misclassifications can still propagate through the pipeline. NGQA substantially reduces human intervention across all pipeline stages, though human validation at key checkpoints is recommended for production environments to ensure correctness and prevent erroneous fixes. As the system incorporates user feedback and learns from validation patterns, reliance on human oversight is expected to diminish further. Finally, evaluation was conducted on open-source repositories without human-annotated fixes, limiting direct comparison with developer-written solutions.

Future work should incorporate datasets with validated fixes and detailed annotations to enable more rigorous benchmarking, explore integration with advanced reasoning models, and develop specialized prompting strategies for different programming paradigms. Seamless CI/CD integration and domain-specific adaptations for specialized software categories would enhance practical applicability. Investigating collaborative human-AI workflows and developing comprehensive evaluation metrics beyond functional correctness represent promising research directions.

Conclusion

This study introduces the NGQA pipeline, a comprehensive framework automating software quality assurance through the intelligent orchestration of static analysis tools and large language models. The six-step pipeline addresses scalable quality assurance challenges with minimal human intervention. Experimental evaluation across 70 repositories spanning seven programming languages demonstrates significant effectiveness: PassRatio increased 16.5% (73.8% to 86.0%), CodeBLEU improved 28.8% (61.2% to 78.8%), and CodeScore improved 24.0% (67.0% to 83.1%). The false positive mitigation agent filtered 598 false positives with an 89.0% F1-score, and the pipeline successfully resolved 83.5% of validated issues. From a practical standpoint, NGQA achieved a 32.6-fold mean acceleration over estimated manual QA effort, completing the full process in 95.9 hours compared to 3,126 person-hours manually. Key contributions include: (1) a near-complete end-to-end quality assurance pipeline integrating static analysis with AI-driven code revision, (2) a novel LCoT framework for structured test case generation using open-source models, (3) comprehensive multi-language evaluation across seven languages, and (4) a dual test case generation strategy enabling four-fold comparative validation. The NGQA pipeline represents a significant step toward largely automated software quality assurance, offering a scalable and privacy-preserving solution for modern software development practices.

References

- [1] Seyed Moein Abtahi and Akramul Azim. 2025. Augmenting Large Language Models with Static Code Analysis for Automated Code Quality Improvements. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, New Orleans, Louisiana, 82–92.
- [2] Seyed Moein Abtahi and Akramul Azim. 2025. WALL: A Web Application for Automated Quality Assurance using Large Language Models. arXiv:2509.09918 [cs.SE] <https://arxiv.org/abs/2509.09918>
- [3] Mohammad Baqar and Rajat Khanda. 2025. The Future of Software Testing: AI-Powered Test Case Generation and Validation. In *Intelligent Computing-Proceedings of the Computing Conference*. Springer, New Delhi, India, 276–300.
- [4] Dirk Beyer. 2025. Advances in automatic software testing: Test-Comp 2025. In *International Conference on Fundamental Approaches to Software Engineering*. Springer Nature Switzerland, Cham, Switzerland, 257–274.
- [5] Barry Boehm, Bradford Clark, Ellis Horowitz, Chris Westland, Ray Madachy, and Richard Selby. 1995. Cost models for future software life cycle processes: COCOMO 2.0. *Annals of software engineering* 1, 1 (1995), 57–94.
- [6] Junkai Chen, Xing Hu, Zhenhao Li, Cuiyun Gao, Xin Xia, and David Lo. 2024. Code search is all you need? improving code suggestions with code search. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, Lisbon, Portugal, 1–13.
- [7] Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. 2025. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567* 1, 1 (2025), 48 pages.
- [8] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. ACM, Melbourne, Australia, 572–576.

- [9] Autumn Clark, Daniel Igbokwe, Samantha Ross, and Minhaz F Zibran. 2024. A quantitative analysis of quality and consistency in ai-generated code. In *2024 7th International Conference on Software and System Engineering (ICoSSE)*. IEEE, Halifax, Canada, 37–41.
- [10] Seyed Shayan Daneshvar, Yu Nong, Xu Yang, Shaowei Wang, and Haipeng Cai. 2024. Exploring RAG-based Vulnerability Augmentation with LLMs. *arXiv preprint arXiv:2408.04125* 1, 1 (2024), 16 pages.
- [11] Nicole Davila and Ingrid Nunes. 2021. A systematic literature review and taxonomy of modern code review. *Journal of Systems and Software* 177 (2021), 110951.
- [12] DeepSeek. 2024. DeepSeek Coder: Open-source code language models. <https://github.com/deepseek-ai>. Accessed: 2025-07-13.
- [13] Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2025. Code-score: Evaluating code generation by learning code execution. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–22.
- [14] Khashayar Etemadi, Nicolas Harrand, Simon Larsén, Haris Adzemovic, Henry Luong Phu, Ashutosh Verma, Fernanda Madeiral, Douglas Wikström, and Martin Monperrus. 2023. Sorald: Automatic Patch Suggestions for SonarQube Static Analysis Violations. *IEEE Transactions on Dependable and Secure Computing* 20, 4 (2023), 2794–2810. doi:10.1109/TDSC.2022.3167316
- [15] Okwudili M Ezeme, Qusay H Mahmoud, and Akramul Azim. 2020. A framework for anomaly detection in time-driven and event-driven processes using kernel traces. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2020), 1–14.
- [16] Lishui Fan, Jiakun Liu, Zhongxin Liu, David Lo, Xin Xia, and Shanning Li. 2024. Exploring the Capabilities of LLMs for Code Change Related Tasks. *arXiv preprint arXiv:2407.02824* 1, 1 (2024), 19 pages.
- [17] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. 2024. Llm agents can autonomously exploit one-day vulnerabilities. *arXiv preprint arXiv:2404.08144* 1, 1 (2024), 18 pages.
- [18] Mohamed Amine Ferrag, Norbert Tihanyi, and Merouane Debbah. 2025. From llm reasoning to autonomous ai agents: A comprehensive review. *arXiv preprint arXiv:2504.19678* 1, 1 (2025), 42 pages.
- [19] Louie Giray. 2023. Prompt engineering with ChatGPT: a guide for academic writers. *Annals of biomedical engineering* 51, 12 (2023), 2629–2633.
- [20] Damian Gneciak and Tomasz Szandala. 2025. Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection. *IEEE Access* 13 (2025), 198410–198422.
- [21] Nam Huynh and Beiyu Lin. 2025. Large Language Models for Code Generation: A Comprehensive Survey of Challenges, Techniques, Evaluation, and Applications. *arXiv preprint arXiv:2503.01245* 1, 1 (2025), 38 pages.
- [22] Mohsen Iranmanesh, Sina Moradi Sabet, Sina Marefat, Ali Javidi Ghasr, Allison Wilson, Iman Sharafaldin, and Mohammad A Tayebi. 2025. ZeroFalse: Improving Precision in Static Analysis with LLMs. *arXiv preprint arXiv:2510.02534* abs/2510.02534, v1 (2025), 1–15.
- [23] Nafis Tanveer Islam, Joseph Khoury, Andrew Seong, Gonzalo De La Torre Parra, Elias Bou-Harb, and Peyman Najafirad. 2024. LLM-Powered Code Vulnerability Repair with Reinforcement Learning and Semantic Reward. *arXiv preprint arXiv:2401.03374* 1, 1 (2024), 17 pages.
- [24] Guilhem Lacombe, David Feliot, Etienne Boespflug, and Marie-Laure Potet. 2024. Combining static analysis and dynamic symbolic execution in a toolchain to detect fault injection vulnerabilities. *Journal of Cryptographic Engineering* 14, 1 (2024), 147–164.
- [25] Chen Ling, Xujiang Zhao, Jiaying Lu, Chengyuan Deng, Can Zheng, Junxiang Wang, Tanmoy Chowdhury, Yun Li, Hejie Cui, Xuchao Zhang, et al. 2023. Domain specialization as the key to make large language models disruptive: A comprehensive survey. *arXiv preprint arXiv:2305.18703* 1, 1 (2023), 50 pages.
- [26] Stephan Lipp, Sebastian Banescu, and Alexander Pretschner. 2022. An Empirical Study on the Effectiveness of Static C Code Analyzers for Vulnerability Detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, New York, NY, USA, 544–555.
- [27] Andrea Lops, Fedelucio Narducci, Azzurra Ragone, Michelantonio Trizio, and Claudio Bartolini. 2025. A system for automated unit test generation using large language models and assessment of generated test suites. In *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, Chicago, Illinois, 29–36.
- [28] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. LLaMA-Reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, San Jose, California, 647–658.
- [29] n8n GmbH. 2025. n8n: Workflow Automation Tool. <https://n8n.io/>. Accessed: 2025-06-13.
- [30] Ollama Contributors. 2025. Ollama: Run LLMs Locally. <https://ollama.com/>. Accessed: 2025-06-13.
- [31] OpenAI. 2023. GPT-4o Technical Report. <https://openai.com/index/hello-gpt-4o/>. Accessed: 2025-06-13.
- [32] OpenAI. 2024. GPT-3.5 Turbo. <https://platform.openai.com/docs/models/gpt-3-5-turbo>.
- [33] Harshad Vijay Pandhare. 2025. Future of Software Test Automation Using AI/ML. *International Journal Of Engineering And Computer Science* 14, 5 (2025), 27159–27182.
- [34] Sebastiano Panichella, Venera Arnaoudova, Massimiliano Di Penta, and Giuliano Antoniol. 2015. Would static analysis tools help developers with code reviews?. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, Montreal, QC, Canada, 161–170.
- [35] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Philadelphia, Pennsylvania, USA, 311–318.
- [36] Debalina Ghosh Paul, Hong Zhu, and Ian Bayley. 2024. Benchmarks and metrics for evaluations of code generation: A critical review. In *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*. IEEE, Arlington, Virginia, 87–94.
- [37] Valentina Piantadosi, Fabiana Fierro, Simone Scalabrino, Alexander Serebrenik, and Rocco Oliveto. 2020. How does code readability change during software evolution? *Empirical Software Engineering* 25 (2020), 5374–5412.
- [38] Alibaba Cloud Qwen Team. 2023. Qwen: Open-source Large Language Models by Alibaba Cloud. <https://github.com/QwenLM>. Accessed: 2025-07-13.
- [39] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [40] Zeeshan Rasheed, Malik Abdul Sami, Muhammad Waseem, Kai-Kristian Kemell, Xiaofeng Wang, Anh Nguyen, Kari Systä, and Pekka Abrahamsson. 2024. AI-powered Code Review with LLMs: Early Results. *arXiv preprint arXiv:2404.18496* 1, 1 (2024), 15 pages.
- [41] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* 1, 1 (2020), 15 pages.
- [42] DeepSeek Research. 2024. DeepSeek Coder: Bridging Retrieval and Generation for Code Language Models. <https://huggingface.co/collections/deepseek-ai/deepseek-coder-65f86f7cd9f7e77b3a56f09f>. Accessed: 2025-07-13.
- [43] Andrew Reynolds and Felix Corrigan. 2024. Improving Real-Time Knowledge Retrieval in Large Language Models with a DNS-Style Hierarchical Query RAG. *Authorea Preprints* 1, 1 (2024), 10 pages.
- [44] Hamidreza Rouzegar and Masoud Makrehchi. 2024. Enhancing Text Classification through LLM-Driven Active Learning and Human Annotation. In *The 18th Linguistic Annotation Workshop (LAW-XVIII) Co-located with EAACL 2024*. Association for Computational Linguistics, Malta, 98.
- [45] SoftwareCost.org. 2026. COCOMO II – Constructive Cost Model. <https://softwarecost.org/tools/COCOMO/>. [https://softwarecost.org/tools/COCOMO/](https://softwarecost.org/tools/COCOMO/Web-based%20implementation%20of%20the%20COCOMO%20II%20estimation%20model)
- [46] SonarSource. 2025. SonarQube. <https://www.sonarsource.com/products/sonarqube/>. Accessed: 2025-06-13.
- [47] Ulligaddala Srinivasarao, Akhil Chaganti, Ajay Mannam, Dhanush Kotrikeline, and Hemanth Katta. 2024. Software Bug Report Detection Methods Based on Machine Learning Techniques. In *2024 2nd International Conference on Advancement in Computation & Computer Technologies (InCACCT)*. IEEE, Bangalore, India, 99–104.
- [48] Jay-Evan J Tevis and John A Hamilton. 2004. Methods for the prevention, detection and removal of software security vulnerabilities. In *Proceedings of the 42nd annual Southeast regional conference*. ACM, Huntsville, Alabama, 197–202.
- [49] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971* <https://arxiv.org/abs/2302.13971>
- [50] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* 1, 1 (2023), 65 pages.
- [51] Asif Kamal Turzo and Amiangshu Bosu. 2024. What makes a code review useful to opendev developers? an empirical investigation. *Empirical Software Engineering* 29, 1 (2024), 6.
- [52] Jiaqi Wang, Zhengliang Liu, Lin Zhao, Zihao Wu, Chong Ma, Sigang Yu, Haixing Dai, Qiushi Yang, Yiheng Liu, Songyao Zhang, et al. 2023. Review of large vision models and visual prompt engineering. *Meta-Radiology* 1, 3 (2023), 100047.
- [53] Yufeng Zhang, Xuepeng Wang, Lingxiang Wu, and Jinqiao Wang. 2025. Enhancing chain of thought prompting in large language models via reasoning patterns. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. AAAI Press, Philadelphia, Pennsylvania, 25985–25993.
- [54] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, and Bin Cui. 2024. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473* 1, 1 (2024), 47 pages.